ASSIGNMENT 3

Submitted By: Ridhi, 1024030528, 3C35

Q1. Exception Hierarchy Demonstration

Demonstrates ArithmeticException, NullPointerException, ArrayIndexOutOfBoundsException, and NumberFormatException, and shows how they can be caught via their parent classes RuntimeException and Exception.

Code:

```java


public class Q1_ExceptionHierarchy {
    public static void main(String[] args) {
        try {
            int x = 10 / 0;
        } catch (ArithmeticException e) {
            System.out.println("ArithmeticException caught: " + e.getMessage());
        }

        try {
            String s = null;
            System.out.println(s.length());
        } catch (NullPointerException e) {
            System.out.println("NullPointerException caught: " + e.getMessage());
        }

        try {
            int[] arr = new int[3];
            arr[5] = 10;
        } catch (ArrayIndexOutOfBoundsException e) {
            System.out.println("ArrayIndexOutOfBoundsException caught: " + e.getMessage());
        }

        try {
            String num = "abc";
            int n = Integer.parseInt(num);
        } catch (NumberFormatException e) {
            System.out.println("NumberFormatException caught: " + e.getMessage());
        }

        try {
            int[] arr2 = new int[2];
            arr2[3] = 1;
        } catch (RuntimeException e) {
            System.out.println("Caught as RuntimeException (parent): " + e.getClass().getSimpleName());
        } catch (Exception e) {
            System.out.println("Caught as Exception (grandparent): " + e.getClass().getSimpleName());
        }
    }
}


```

Output:

```
ArithmeticException caught: / by zero
NullPointerException caught: Cannot invoke "String.length()" because "<local1>" is null
ArrayIndexOutOfBoundsException caught: Index 5 out of bounds for length 3
NumberFormatException caught: For input string: "abc"
Caught as RuntimeException (parent): ArrayIndexOutOfBoundsException

=== Code Execution Successful ===
```

Q2. Nested Try-Catch Blocks

Contains nested try-catch blocks that generate different exceptions in the inner and outer blocks, with finally blocks in both.

Code:

```java


public class Q2_NestedTryCatch {
    public static void main(String[] args) {
        try {
            System.out.println("Outer try block started");
            int[] arr = {1, 2, 3};

            try {
                System.out.println("Inner try block started");
                int result = arr[1] / 0;
            } catch (ArithmeticException e) {
                System.out.println("Inner catch: ArithmeticException - " + e.getMessage());
            } finally {
                System.out.println("Inner finally block executed");
            }

            int value = arr[10];
        } catch (ArrayIndexOutOfBoundsException e) {
            System.out.println("Outer catch: ArrayIndexOutOfBoundsException - " + e.getMessage());
        } finally {
            System.out.println("Outer finally block executed");
        }

        System.out.println("Program continues after nested try-catch");
    }
}


```

Output:

```
Outer try block started
Inner try block started
Inner catch: ArithmeticException - / by zero
Inner finally block executed
Outer catch: ArrayIndexOutOfBoundsException - Index 10 out of bounds for length 3
Outer finally block executed
Program continues after nested try-catch

=== Code Execution Successful ===
```

Q3. Calculator with Exception Handling

Accepts two numbers and an operator, and handles Division by zero, Invalid numeric input, and Invalid operators using separate exception handlers. Shown below are four sample runs.

Code:

```java
import java.util.Scanner;

public class Q3_Calculator {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter first number: ");
        String num1 = sc.next();
        System.out.print("Enter second number: ");
        String num2 = sc.next();
        System.out.print("Enter operator (+, -, *, /): ");
        String op = sc.next();

        try {
            double a = Double.parseDouble(num1);
            double b = Double.parseDouble(num2);
            double result;

            switch (op) {
                case "+":
                    result = a + b;
                    break;
                case "-":
                    result = a - b;
                    break;
                case "*":
                    result = a * b;
                    break;
                case "/":
                    if (b == 0) {
                        throw new ArithmeticException("Division by zero is not allowed");
                    }
                    result = a / b;
                    break;
                default:
                    throw new IllegalArgumentException("Invalid operator: " + op);
            }
            System.out.println("Result: " + result);

        } catch (NumberFormatException e) {
            System.out.println("Error: Invalid numeric input");
        } catch (ArithmeticException e) {
            System.out.println("Error: " + e.getMessage());
        } catch (IllegalArgumentException e) {
            System.out.println("Error: " + e.getMessage());
        } finally {
            sc.close();
        }
    }
}
```

Run 1: Division by zero (10 / 0)

```
Input: 10 0 /

Enter first number: Enter second number: Enter operator (+, -, *, /): Error: Division by zero is not allowed

=== Code Execution Successful ===
```

Run 2: Invalid numeric input (abc + 5)

```
Input: abc 5 +

Enter first number: Enter second number: Enter operator (+, -, *, /): Error: Invalid numeric input

=== Code Execution Successful ===
```

Run 3: Invalid operator (10 % 5)

```
Input: 10 5 %

Enter first number: Enter second number: Enter operator (+, -, *, /): Error: Invalid operator: %

=== Code Execution Successful ===
```

Run 4: Valid operation (10 * 5)

```
Input: 10 5 *

Enter first number: Enter second number: Enter operator (+, -, *, /): Result: 50.0

=== Code Execution Successful ===
```

Q4. Student Result Processing with Custom Exception

Accepts marks for 5 subjects and validates that each is between 0 and 100 using a custom InvalidMarksException. Total, percentage, and grade are calculated only when all input is valid.

Code:

```java
import java.util.Scanner;

class InvalidMarksException extends Exception {
    public InvalidMarksException(String message) {
        super(message);
    }
}

public class Q4_StudentResult {
    static void validateMarks(int marks) throws InvalidMarksException {
        if (marks < 0 || marks > 100) {
            throw new InvalidMarksException("Invalid marks: " + marks + ". Marks must be between 0 and 100");
        }
    }

    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        int n = 5;
        int[] marks = new int[n];
        boolean valid = true;

        for (int i = 0; i < n; i++) {
            System.out.print("Enter marks for subject " + (i + 1) + ": ");
            marks[i] = sc.nextInt();
            try {
                validateMarks(marks[i]);
            } catch (InvalidMarksException e) {
                System.out.println(e.getMessage());
                valid = false;
            }
        }

        if (valid) {
            int total = 0;
            for (int m : marks) {
                total += m;
            }
            double percentage = (double) total / n;
            char grade;
            if (percentage >= 90) grade = 'A';
            else if (percentage >= 75) grade = 'B';
            else if (percentage >= 60) grade = 'C';
            else if (percentage >= 40) grade = 'D';
            else grade = 'F';

            System.out.println("Total: " + total);
            System.out.println("Percentage: " + percentage);
            System.out.println("Grade: " + grade);
        } else {
            System.out.println("Result cannot be processed due to invalid marks entered above");
        }
        sc.close();
    }
}
```

Run 1: Invalid marks entered (120, -5)

```
Input: 80 90 70 120 -5

Enter marks for subject 1: Enter marks for subject 2: Enter marks for subject 3: Enter marks for subject 4:
Invalid marks: 120. Marks must be between 0 and 100
Enter marks for subject 5: Invalid marks: -5. Marks must be between 0 and 100
Result cannot be processed due to invalid marks entered above

=== Code Execution Successful ===
```

Run 2: All marks valid

```
Input: 80 90 70 60 85

Enter marks for subject 1: Enter marks for subject 2: Enter marks for subject 3: Enter marks for subject 4:
Enter marks for subject 5: Total: 385
Percentage: 77.0
Grade: B

=== Code Execution Successful ===
```

Q5. Banking Application with Custom Exceptions

Supports deposit and withdrawal with custom InsufficientBalanceException, InvalidAmountException, and AccountNotFoundException, demonstrating exception propagation between methods.

Code:

```java


class InsufficientBalanceException extends Exception {
    public InsufficientBalanceException(String message) {
        super(message);
    }
}

class InvalidAmountException extends Exception {
    public InvalidAmountException(String message) {
        super(message);
    }
}

class AccountNotFoundException extends Exception {
    public AccountNotFoundException(String message) {
        super(message);
    }
}

class BankAccount {
    private int accountNumber;
    private double balance;

    public BankAccount(int accountNumber, double balance) {
        this.accountNumber = accountNumber;
        this.balance = balance;
    }

    public int getAccountNumber() {
        return accountNumber;
    }

    public double getBalance() {
        return balance;
    }

    public void deposit(double amount) throws InvalidAmountException {
        checkAmount(amount);
        balance += amount;
    }

    public void withdraw(double amount) throws InvalidAmountException, InsufficientBalanceException {
        checkAmount(amount);
        if (amount > balance) {
            throw new InsufficientBalanceException("Insufficient balance for withdrawal of " + amount);
        }
        balance -= amount;
    }

    private void checkAmount(double amount) throws InvalidAmountException {
        if (amount <= 0) {
            throw new InvalidAmountException("Amount must be positive: " + amount);
        }
    }
}

public class Q5_BankingApp {
    static BankAccount findAccount(BankAccount[] accounts, int accNo) throws AccountNotFoundException {
        for (BankAccount acc : accounts) {
            if (acc.getAccountNumber() == accNo) {
                return acc;
```

```
        }
        throw new AccountNotFoundException("Account number " + accNo + " not found");
    }

    public static void main(String[] args) {
        BankAccount[] accounts = {
            new BankAccount(101, 5000),
            new BankAccount(102, 2000)
        };

        try {
            BankAccount acc1 = findAccount(accounts, 101);
            acc1.deposit(1000);
            System.out.println("Deposited 1000. Balance: " + acc1.getBalance());

            acc1.withdraw(3000);
            System.out.println("Withdrew 3000. Balance: " + acc1.getBalance());

            acc1.withdraw(10000);
        } catch (InvalidAmountException | InsufficientBalanceException | AccountNotFoundException e) {
            System.out.println("Transaction failed: " + e.getMessage());
        }

        try {
            BankAccount acc2 = findAccount(accounts, 999);
        } catch (AccountNotFoundException e) {
            System.out.println("Error: " + e.getMessage());
        }

        try {
            BankAccount acc3 = findAccount(accounts, 102);
            acc3.deposit(-500);
        } catch (InvalidAmountException | AccountNotFoundException e) {
            System.out.println("Error: " + e.getMessage());
        }
    }
}
```

Output:

```
Deposited 1000. Balance: 6000.0
Withdrew 3000. Balance: 3000.0
Transaction failed: Insufficient balance for withdrawal of 10000.0
Error: Account number 999 not found
Error: Amount must be positive: -500.0

=== Code Execution Successful ===
```

Q6. Login System with Exception-Based Validation

Uses custom InvalidUsernameException, InvalidPasswordException, and AccountLockedException. The account is locked after a fixed number (3) of failed login attempts, using try-catch-finally.

Code:

```java


class InvalidUsernameException extends Exception {
    public InvalidUsernameException(String message) {
        super(message);
    }
}

class InvalidPasswordException extends Exception {
    public InvalidPasswordException(String message) {
        super(message);
    }
}

class AccountLockedException extends Exception {
    public AccountLockedException(String message) {
        super(message);
    }
}

public class Q6_LoginSystem {
    static final String VALID_USERNAME = "admin";
    static final String VALID_PASSWORD = "pass123";
    static final int MAX_ATTEMPTS = 3;
    static int attemptCount = 0;
    static boolean locked = false;

    static void login(String username, String password) throws InvalidUsernameException, InvalidPasswordExcepti
        if (locked) {
            throw new AccountLockedException("Account is locked due to too many failed attempts");
        }
        try {
            if (!username.equals(VALID_USERNAME)) {
                throw new InvalidUsernameException("Username '" + username + "' does not exist");
            }
            if (!password.equals(VALID_PASSWORD)) {
                throw new InvalidPasswordException("Incorrect password");
            }
            System.out.println("Login successful for user: " + username);
            attemptCount = 0;
        } catch (InvalidUsernameException | InvalidPasswordException e) {
            attemptCount++;
            if (attemptCount >= MAX_ATTEMPTS) {
                locked = true;
            }
            throw e;
        } finally {
            System.out.println("Attempt " + attemptCount + " of " + MAX_ATTEMPTS + " recorded");
        }
    }

    public static void main(String[] args) {
        String[][] attempts = {
            {"user1", "pass123"},
            {"admin", "wrongpass"},
            {"admin", "wrong2"},
            {"admin", "pass123"}
        };

        for (String[] attempt : attempts) {
            try {
                login(attempt[0], attempt[1]);
            } catch (InvalidUsernameException e) {
                System.out.println("Login failed: " + e.getMessage());
            } catch (InvalidPasswordException e) {
```

```
                System.out.println("Login failed: " + e.getMessage());
            } catch (AccountLockedException e) {
                System.out.println("Login failed: " + e.getMessage());
            }
        }
    }
}
```

Output:

```
Attempt 1 of 3 recorded
Login failed: Username 'user1' does not exist
Attempt 2 of 3 recorded
Login failed: Incorrect password
Attempt 3 of 3 recorded
Login failed: Incorrect password
Login failed: Account is locked due to too many failed attempts

=== Code Execution Successful ===
```

Q7. Shopping Cart with a Complete Exception Hierarchy

Implements the ApplicationException hierarchy (ProductException, PaymentException, OrderException and their subclasses) for a small shopping cart application supporting add/remove, search, and checkout operations.

Code:

```java
import java.util.ArrayList;
import java.util.List;

class ApplicationException extends Exception {
    public ApplicationException(String message) {
        super(message);
    }
}

class ProductException extends ApplicationException {
    public ProductException(String message) {
        super(message);
    }
}

class ProductNotFoundException extends ProductException {
    public ProductNotFoundException(String message) {
        super(message);
    }
}

class OutOfStockException extends ProductException {
    public OutOfStockException(String message) {
        super(message);
    }
}

class PaymentException extends ApplicationException {
    public PaymentException(String message) {
        super(message);
    }
}

class InvalidPaymentException extends PaymentException {
    public InvalidPaymentException(String message) {
        super(message);
    }
}

class InsufficientFundsException extends PaymentException {
    public InsufficientFundsException(String message) {
        super(message);
    }
}

class OrderException extends ApplicationException {
    public OrderException(String message) {
        super(message);
    }
}

class EmptyCartException extends OrderException {
    public EmptyCartException(String message) {
        super(message);
```

```
    }
}

class Product {
    String name;
    double price;
    int stock;

    public Product(String name, double price, int stock) {
        this.name = name;
        this.price = price;
        this.stock = stock;
    }
}

class Cart {
    List<Product> items = new ArrayList<>();

    void addItem(Product p, int qty) throws OutOfStockException {
        if (p.stock < qty) {
            throw new OutOfStockException(p.name + " is out of stock. Available: " + p.stock);
        }
        p.stock -= qty;
        items.add(p);
    }

    double getTotal() {
        double total = 0;
        for (Product p : items) {
            total += p.price;
        }
        return total;
    }

    void checkout(double amountPaid) throws EmptyCartException, InsufficientFundsException {
        if (items.isEmpty()) {
            throw new EmptyCartException("Cannot checkout an empty cart");
        }
        double total = getTotal();
        if (amountPaid < total) {
            throw new InsufficientFundsException("Amount paid " + amountPaid + " is less than total " + total)
        }
        System.out.println("Checkout successful. Total paid: " + amountPaid + ", Change: " + (amountPaid - tot
        items.clear();
    }
}

public class Q7_ShoppingCart {
    static Product findProduct(Product[] products, String name) throws ProductNotFoundException {
        for (Product p : products) {
            if (p.name.equalsIgnoreCase(name)) {
                return p;
            }
        }
        throw new ProductNotFoundException("Product '" + name + "' not found");
    }

    static void validatePayment(String method) throws InvalidPaymentException {
        if (!method.equals("CARD") && !method.equals("CASH")) {
            throw new InvalidPaymentException("Invalid payment method: " + method);
        }
    }

    public static void main(String[] args) {
        Product[] products = {
            new Product("Laptop", 55000, 2),
            new Product("Mouse", 500, 0),
            new Product("Keyboard", 1200, 5)
        };

        Cart cart = new Cart();

        try {
```

```
        try {
            Product p1 = findProduct(products, "Laptop");
            cart.addItem(p1, 1);
            System.out.println("Added Laptop to cart");

            Product p2 = findProduct(products, "Mouse");
            cart.addItem(p2, 1);
        } catch (ProductNotFoundException | OutOfStockException e) {
            System.out.println("Error: " + e.getMessage());
        }

        try {
            validatePayment("UPI");
        } catch (InvalidPaymentException e) {
            System.out.println("Error: " + e.getMessage());
        }

        try {
            cart.checkout(30000);
        } catch (EmptyCartException | InsufficientFundsException e) {
            System.out.println("Checkout failed: " + e.getMessage());
        }

        try {
            cart.checkout(60000);
        } catch (EmptyCartException | InsufficientFundsException e) {
            System.out.println("Checkout failed: " + e.getMessage());
        }

        try {
            cart.checkout(100000);
        } catch (ApplicationException e) {
            System.out.println("Checkout failed (caught as ApplicationException): " + e.getMessage());
        }
    }
}
```

Output:

```
Added Laptop to cart
Error: Mouse is out of stock. Available: 0
Error: Invalid payment method: UPI
Checkout failed: Amount paid 30000.0 is less than total 55000.0
Checkout successful. Total paid: 60000.0, Change: 5000.0
Checkout failed (caught as ApplicationException): Cannot checkout an empty cart

=== Code Execution Successful ===
```